UNIVERSITY OF KARACHI

Department of Computer Science (UBIT)

LAB NO. 06

Operating Systems (SE-458)

SUBMITTED BY:	SUBMITTED TO:
Shaikh Mahad Ud Din Seat No: EB25210106113 Section: BSSE A Evening	**Ms. Nazish Imran** Faculty of Computer Science

---

Date of Submission: 20 October 2026

LAB NO. 06 - DECISION CONTROL AND LOOPS IN BASH

PART A: DECISION CONTROL AND LOOPS IN BASH

Objective

To use conditions, loops and input validation in Bash, and develop a student marksheet program that calculates totals, percentages, grades and results.

1. Program Control Flow

Control flow determines the order in which a program executes its instructions.

- Sequence: Instructions run one after another.
- Selection: A condition determines which instructions run.
- Iteration: A loop repeats a set of instructions.

2. Decision Statements

if Statement

An if statement executes a block only when its condition is true.

```shell
if [ "$marks" -ge 40 ]; then
    echo "PASS"
fi
```

The then keyword begins the block, and fi ends it.

if-else Statement

An if-else statement selects between two alternatives.

```shell
if [ "$marks" -ge 40 ]; then
    echo "PASS"
else
    echo "FAIL"
fi
```

if-elif-else Statement

This structure checks several conditions in order. Only the first matching branch executes.

```shell
if [ "$percentage" -ge 80 ]; then
    grade=A
elif [ "$percentage" -ge 70 ]; then
    grade=B
elif [ "$percentage" -ge 60 ]; then
    grade=C
elif [ "$percentage" -ge 50 ]; then
    grade=D
else
    grade=F
fi
```

Nested if Statement

A nested if places one decision inside another.

```shell
if [ "$marks" -ge 40 ]; then
    if [ "$marks" -ge 80 ]; then
        echo "PASS with distinction"
    else
        echo "PASS"
    fi
else
    echo "FAIL"
fi
```

3. Comparison and Logical Operators

Numeric Comparison Operators

- -eq: Equal to.
- -ne: Not equal to.
- -gt: Greater than.
- -ge: Greater than or equal to.
- -lt: Less than.
- -le: Less than or equal to.

Logical Operators

- &&: AND. Both conditions must be true.
- ||: OR. At least one condition must be true.
- !: NOT. Reverses the result of a condition.

Test Forms in Bash

- [ ]: A test command supported by POSIX shells.
- [[ ]]: Bash conditional syntax that supports pattern matching and regular expressions.
- (( )): Evaluates arithmetic conditions.
- $(( )): Calculates an integer value.

4. The case Statement

A case statement selects an action by matching a value against patterns.

```Shell
case "$choice" in
    1) echo "Enter details" ;;
    2) echo "Display marksheet" ;;
    3) echo "Exit" ;;
    *) echo "Invalid choice" ;;
esac
```

5. Loops

for Loop

A for loop repeats instructions for a list of values or a known number of iterations.

```Shell
for i in 1 2 3 4 5; do
    echo "$i"
done
```

Arithmetic form:

```Shell
for ((i=1; i<=5; i++)); do
    echo "$i"
done
```

while Loop

A while loop repeats while its condition is true.

```shell
i=1
while [ "$i" -le 5 ]; do
    echo "$i"
    i=$((i + 1))
done
```

until Loop

An until loop repeats while its condition is false.

```shell
i=1
until [ "$i" -gt 5 ]; do
    echo "$i"
    i=$((i + 1))
done
```

6. break and continue

- break: Ends the nearest enclosing loop immediately.
- continue: Skips the remaining statements in the current iteration.

7. Arrays

An array stores several values under one variable name.

```shell
subjects=("Programming" "Operating Systems" "Mathematics")
echo "${subjects[0]}"
echo "${#subjects[@]}"
```

8. Input Validation

Input should be checked before arithmetic or comparison. Student name and seat number must not be empty. Marks must be whole numbers from 0 to 100.

9. Total, Percentage and Output Formatting

The total is the sum of all subject marks.

Percentage = (Total obtained marks * 100) / Total possible marks.

The printf command produces consistent output using format specifiers like %d and %s.

10. Marksheet Rules

- A: 80% and above.
- B: 70% to below 80%.
- C: 60% to below 70%.
- D: 50% to below 60%.
- F: Below 50%.

Overall PASS requires at least 40 marks in every subject.

PART B: MARKSHEET PROGRAM AND SELECTED CLASSWORK

Task 1: Develop a Student Marksheet Program

Write a Bash program that accepts a student's name, seat number and marks in five subjects. Validate the input, calculate the total and percentage, assign a grade, and display the result for each subject and the overall result.

Program Requirements

- Accept marks from 0 to 100 in each subject.
- Ask again when the name, seat number or marks are invalid.
- Store subject names and marks in arrays.
- Calculate the total out of 500 and percentage to two decimal places.
- Assign the percentage grade using if-elif-else.
- Display FAIL overall if any subject has fewer than 40 marks.

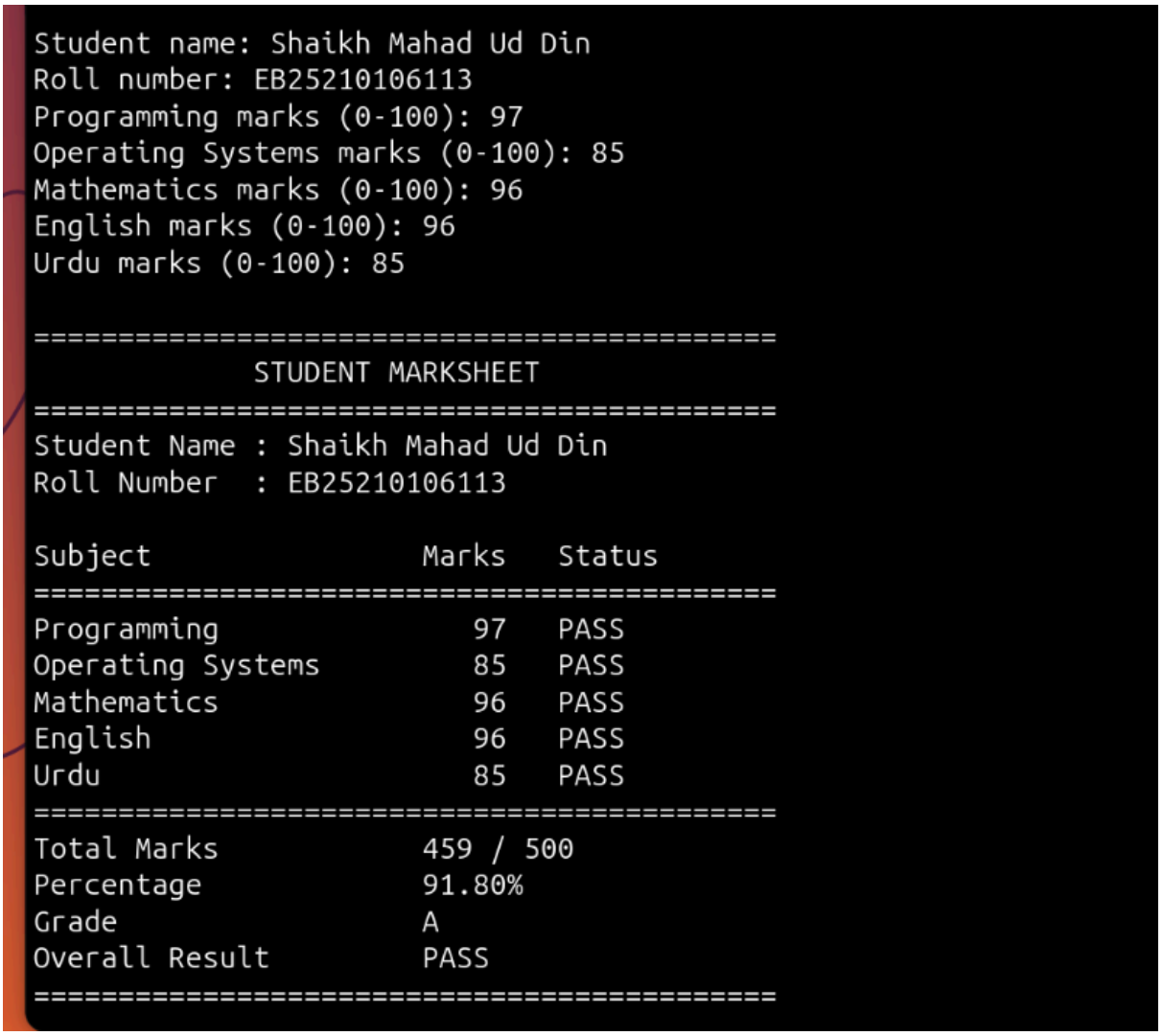
Commands

Shell

```
#!/bin/bash
while true; do
    read -r -p "Student name: " name || exit 1
    [[ $name =~ [^[:space:]] ]] && break
    echo "Name cannot be empty."
done
while true; do
    read -r -p "Seat number: " seat || exit 1
    [[ $seat =~ [^[:space:]] ]] && break
    echo "Seat number cannot be empty."
done
subjects=("Programming" "Operating Systems" "Mathematics" "English" "Urdu")
marks=()
total=0
failed=0
```

```
for subject in "${subjects[@]}"; do
    while true; do
        read -r -p "$subject marks (0-100): " m || exit 1
        if [[ $m =~ ^[0-9]{1,3}$ ]]; then
            m=$((10#$m))
            if (( m <= 100 )); then break; fi
        fi
        echo "Invalid marks. Enter a whole number from 0 to 100."
    done
    marks+=("$m")
    total=$((total + m))
    if (( m < 40 )); then failed=1; fi
done
maximum=$((${#subjects[@]} * 100))
pct=$((total * 10000 / maximum))
if (( pct >= 8000 )); then grade=A
elif (( pct >= 7000 )); then grade=B
elif (( pct >= 6000 )); then grade=C
elif (( pct >= 5000 )); then grade=D
else grade=F
fi
result=PASS
if (( failed == 1 )); then result=FAIL; fi
printf '\nSTUDENT MARKSHEET\n'
printf 'Name: %s\nSeat number: %s\n' "$name" "$seat"
printf '\n%-20s %5s %7s\n' "Subject" "Marks" "Status"
for ((i=0; i<${#subjects[@]}; i++)); do
    status=PASS
    if (( marks[i] < 40 )); then status=FAIL; fi
    printf '%-20s %5d %7s\n' "${subjects[i]}" "${marks[i]}" "$status"
done
printf '\nTotal: %d / %d\n' "$total" "$maximum"
printf 'Percentage: %d.%02d%%\n' "$((pct / 100))" "$((pct % 100))"
printf 'Grade: %s\nOverall result: %s\n' "$grade" "$result"
```

Output



```
Student name: Shaikh Mahad Ud Din
Roll number: EB25210106113
Programming marks (0-100): 97
Operating Systems marks (0-100): 85
Mathematics marks (0-100): 96
English marks (0-100): 96
Urdu marks (0-100): 85

============================================
             STUDENT MARKSHEET
============================================
Student Name : Shaikh Mahad Ud Din
Roll Number  : EB25210106113

Subject                Marks   Status
============================================
Programming               97   PASS
Operating Systems         85   PASS
Mathematics               96   PASS
English                   96   PASS
Urdu                      85   PASS
============================================
Total Marks            459 / 500
Percentage             91.80%
Grade                  A
Overall Result         PASS
============================================
```

Classwork Exercise 1: Positive, Negative or Zero

Write a Bash program that reads an integer and determines whether it is positive, negative or zero.

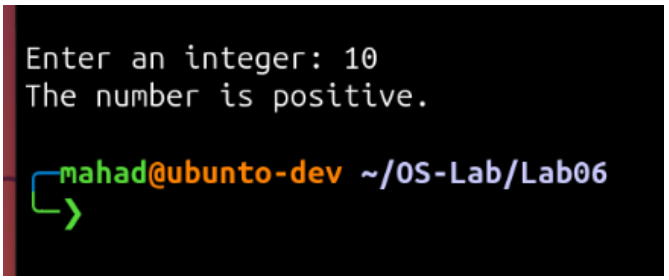
Commands

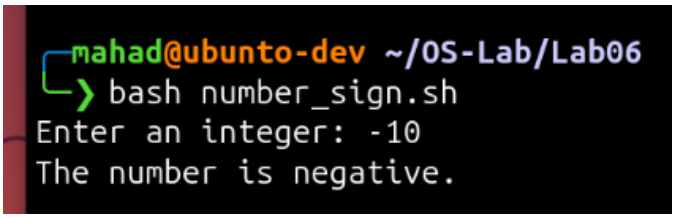
Shell

```
#!/bin/bash
read -r -p "Enter an integer: " number
if (( number > 0 )); then
    echo "The number is positive."
elif (( number < 0 )); then
    echo "The number is negative."
else
```

```
    echo "The number is zero."
fi
```

Output





Classwork Exercise 4: Largest of Three Numbers

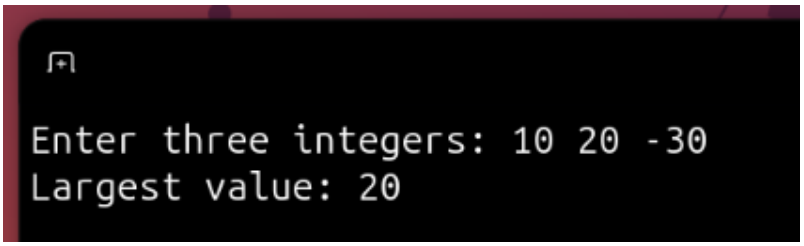
Write a Bash program that reads three integers and displays the largest value.

Commands

Shell

```
#!/bin/bash
read -r -p "Enter three integers: " a b c
if (( a >= b && a >= c )); then
    largest=$a
elif (( b >= a && b >= c )); then
    largest=$b
else
    largest=$c
fi
echo "Largest value: $largest"
```

Output



Classwork Exercise 6: Multiplication Table

Write a Bash program that reads an integer and displays its multiplication table from 1 to 10.

Commands

```shell
#!/bin/bash
read -r -p "Enter an integer: " number
for ((i=1; i<=10; i++)); do
    echo "$number x $i = $((number * i))"
done
```

Output

